AI Hardware - Compiler

By: Mikhail Golovenchits, Sahil Patel, Heng-I Chu, Vihaan Chinthakindi
Mentors: Timothy Hein, Sooraj Chetput, Dr. Mark C. Johnson

Overview

- Any processor needs instructions to function, and writing those instructions in pure assembly is inefficient
- A **custom compiler**, based on PPCI (Pure-Python Compiler), translates high-level C code into the **hardware-specific ISA**.
- The compiler structures instructions into efficient **Very Long Instruction Word (VLIW)** packets, by tracking latency and dependencies. Addressing this in the compiler is **significantly cheaper** than configuring hardware scheduling.
- Overall, the proposed compiler offers an easy-to-use and efficient solution by significantly speeding up development of AI kernels.

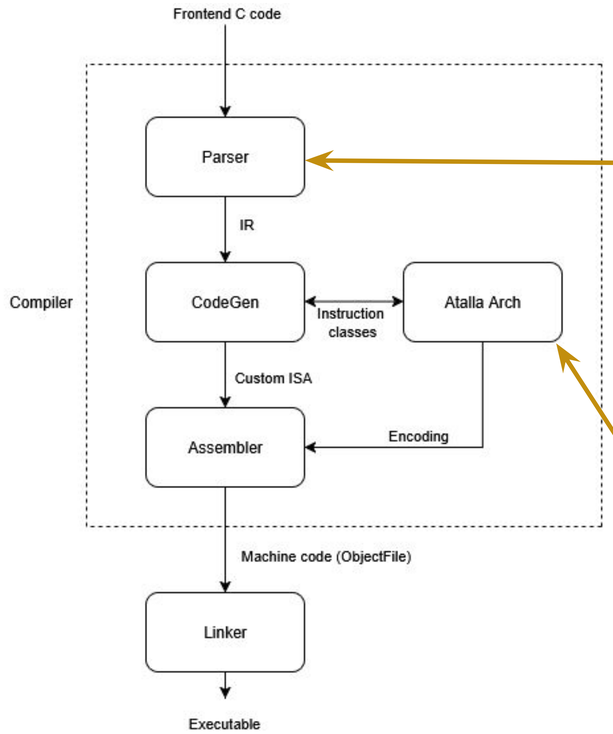
Packetization

- Generates **tightly packed VLIWs** with dependency timing.
- Packetization happens *after* the CodeGen phase.
- Eliminate illegal packets by checking all data, control, and memory hazards.
- **Improves ILP** (instruction-level parallelism) while keeping hardware constraints by placing all ready-independent ops in the earliest possible cycle.

Assembly Code to Instruction Entries (operands, type, latency)

Dependency graph (Each instruction's earliest safe start cycle)

Greedily fill VLIW packets (up to 4 ops) while enforcing:
- RAW/WAR/WAW hazards with single-LSU restriction.
- No control-flow mixing.
- Register/FU usage limits (scoreboarding).




```
addi x1, x0, 5
addi x2, x1, 3
mul  x3, x1, x2
sw   x1, 0(x2)
lw   x4, 0(x2)
sub  x2, x4, x5
sub  x4, x3, x2
add  x5, x4, x1
or   x6, x1, x2
L1:
add  x11, x12, x13
add  x12, x13, x14
add  x14, x15, x16
```

Instruction	Latency
Addi	1
Add	1
Sub	1
Mul	3
Or	1
Lw	3
Sw	1

```
Packet 0:
   addi x1, x0, 5
   add  x11, x12, x13
   add  x14, x15, x16
   nop
Packet 1:
   addi x2, x1, 3
   add  x12, x13, x14
   nop
   nop
Packet 2:
   mul  x3, x1, x2
   sw   x1, 0(x2)
   nop
   nop
Packet 3:
   lw   x4, 0(x2)
   nop
   nop
   nop
```

```
Packet 4:
   sub  x2, x4, x5
   nop
   nop
   nop
Packet 5:
   sub  x4, x3, x2
   or   x6, x1, x2
   nop
   nop
Packet 6:
   add  x5, x4, x1
   nop
   nop
   nop
```

Frontend

- C-like frontend extended with custom intrinsic functions for GEMM, SDMA, Vector-Load, etc.
- **"Vec" data type** mapped onto vector registers.
- Implemented **bfloat16 arrays** by modifying ir.float to support 16-bits elements.

Backend

- Implemented custom Atalla ISA Architecture.
- Created custom Scalar, Vector, GEMM instructions for backend architecture.
- Built ISA patterns for each instruction within the ISA to generate assembly from IR.

Roadmap

- Fully implement linker to generate executables.
- Complete ISA patterns for vector instructions.
- Integrate packetization scripts into the compiler.
- Test assembly using the Atalla functional simulator.